

Quantum AI Labs - QML Track
Introduced by iQafe and Bibliotheca Alexandrina, in collaboration
with EgQCC

Quantum AutoEncoders

Final Project

Sri Harsha Nallan Chakravarthula

Abstract

Quantum autoencoders (QAEs) adapt the core idea of autoencoding-compressing structured data through a bottleneck while preserving relevant information-to quantum states and quantum circuits. This report studies the QAE objective and training loop, clarifies how “latent” and “trash” subsystems are used to implement compression under unitary constraints, and evaluates realism under near-term (NISQ) hardware limits. A small statevector simulation demonstrates a simplified two-qubit-to-one-qubit compression task, and the method is critically compared to classical autoencoders, PCA-style baselines, and information-theoretic quantum compression. [1, 2]

Contents

1	Problem Definition	2
2	Algorithm Explanation	2
3	Hybrid Architecture	3
4	Step-by-Step Pipeline	3
5	Implementation	4
6	Classical Comparison	6
7	Limitations and Challenges	6
8	Critical Evaluation	6
9	Quantum Advantage Discussion	7
10	Conclusion	7

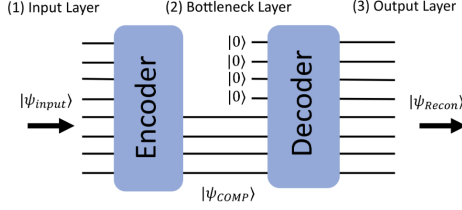


Figure 1: Quantum Autoencoder

1 Problem Definition

A quantum autoencoder is primarily a *quantum data compression* method: given an ensemble of quantum states on n qubits, it aims to learn an encoding circuit that maps these states into a smaller latent register of $m < n$ qubits while pushing the remaining “trash” register toward a fixed reference state (often $|0 \cdots 0\rangle$), so the trash can be discarded with minimal information loss. [1, 3]

In machine-learning terms, this most naturally fits *unsupervised learning* / *dimensionality reduction*, with variants used for denoising and (less canonically) anomaly detection when classical data are embedded into quantum states. [4, 5]

2 Algorithm Explanation

A QAE learns parameters θ of a unitary encoder $U(\theta)$ with the target behavior

$$U(\theta)|\psi\rangle \approx |\phi(\psi)\rangle_{\text{latent}} \otimes |0 \cdots 0\rangle_{\text{trash}}.$$

If the trash subsystem is close to the reference state and effectively decoupled, discarding it preserves (most of) the information needed for reconstruction; decoding is implemented via $U(\theta)^\dagger$ (plus re-initializing the trash in $|0 \cdots 0\rangle$). [1, 3]

Data encoding into the quantum system. For quantum-native QAEs, the “data” are already quantum states prepared by a quantum routine. For classical data, an embedding (angle/basis/amplitude) must be chosen; amplitude-style loading is information-dense but can be costly for generic vectors, motivating careful discussion of input models and state-preparation overhead. [6, 7]

Circuit and measurement. Practical implementations often train using a cost based on the trash subsystem: either (i) maximizing $P(\text{trash} = |0 \cdots 0\rangle)$ (“junk occupation” minimization) or (ii) estimating a fidelity proxy between trash and a reference using overlap tests. [8, 1] Gradients (when used) can be estimated with analytic circuit-gradient methods such as parameter-shift rules. [9]

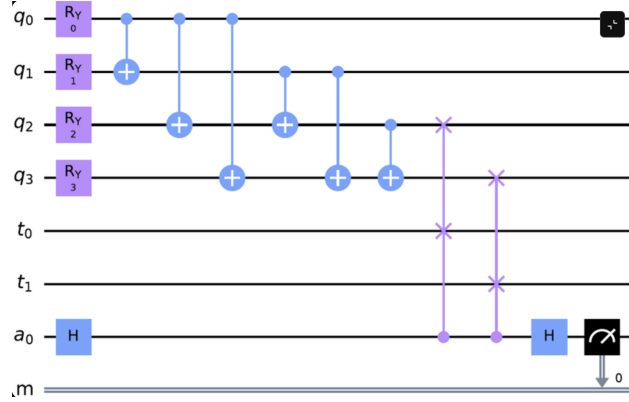


Figure 2: Circuit diagram

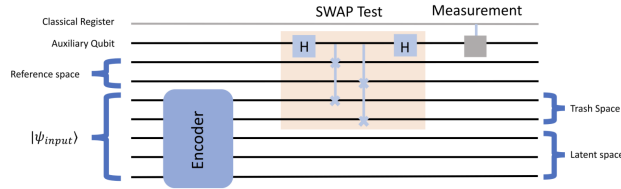


Figure 3: QAE with SWAP Test

3 Hybrid Architecture

QAEs are typically trained in a hybrid loop: a quantum circuit evaluates a loss function via measurements, and a classical optimizer updates circuit parameters. This division is not optional in near-term practice because optimization, batching, stopping criteria, and hyperparameter selection are handled classically while model evaluation requires executing quantum circuits. [1, 2]

The classical components sit in places you’d expect from variational algorithms: choosing the ansatz and embedding, batching training examples, estimating losses from finite-shot measurement outcomes, and running an optimizer (gradient-free methods like COBYLA are especially common in noisy objectives). When gradients are used, analytic gradient estimators like the parameter-shift rule allow gradient evaluation via additional circuit executions rather than numerical differentiation. The quantum components include state preparation/encoding, execution of the parameterized circuit ($U(\theta)$), and measurement routines for the cost function (SWAP test or trash-qubit measurements). The quantum part is typically trainable (parameters (θ)), while the embedding and reference state might be fixed

[9, 10, 11]

4 Step-by-Step Pipeline

Data encoding

Prepare each training input as a quantum state ($|\psi_i\rangle$). For quantum-native data this is “just” state preparation; for classical data this is the embedding step (angle/basis/amplitude encoding), which may be the dominant cost.

Quantum processing

Apply the parameterized encoder ($U(\theta)$) to produce (ideally) a latent subsystem carrying the information and a trash subsystem near a reference state.

Measurement

Estimate a proxy for compression quality, typically by:

SWAP test fidelity between trash and reference (via ancilla statistics), or measuring trash qubits directly and maximizing ($P(\text{trash} = 0 \cdots 0)$) (“minimize junk occupation”).

Classical post-processing

Convert bitstring counts (shots) into a scalar loss estimate; optionally track variance because shot noise affects optimization stability.

Training / optimization

Update (θ) using a classical optimizer. Derivative-free optimizers like COBYLA are commonly used for noisy objectives and are historically associated with constrained optimization by linear approximations. If gradients are used, evaluate them with circuit-based estimators (parameter-shift and related analytic-gradient techniques).

5 Implementation

Implementation summary

This implementation demonstrates a simplified 2-qubit $\rightarrow$ 1-qubit QAE trained to compress a family of quantum states that is provably compressible (they live in a 2D subspace). The goal is to make the mechanics explicit: parameterized encoding, trash-qubit objective, classical optimization, and a reconstruction fidelity check. The compressibility condition mirrors the “latent dimension span dimension” criterion used in lossless-QAE analyzes and experiments.

Training data (quantum dataset): random states restricted to ($\text{span}|00\rangle, |11\rangle$): $[|\psi(\theta, \phi)\rangle = \cos(\theta)|00\rangle + e^{i\phi}\sin(\theta)|11\rangle]$. This set spans at most a 2D subspace, so it is compressible into a 1-qubit latent space in principle.

Ansatz: $((U_0 \otimes U_1), \text{CNOT}, (V_0 \otimes V_1))$ with Euler-angle single-qubit gates (12 parameters total).

Loss: $(1 - \mathbb{E}[P(\text{trash} = |0\rangle)])$. This is the “junk occupation” idea: push trash toward a fixed reference so it can be discarded.

Optimizer: COBYLA (derivative-free). Reconstruction check: encode $\rightarrow$ trace out trash $\rightarrow$ append ($|0\rangle$) $\rightarrow$ decode using ($U^\dagger$) $\rightarrow$ compute state fidelity with the original input.

Table 1: QAE experiment results

Phase	Mean $P(\text{trash}=0)$	Mean Recon. fidelity train	Objective
Before training	0.526701404888939	0.397907660892159	0.473298595111061
After training	0.999999999995011	0.999999999993689	4.98923125036299E-12
Before testing	0.533266466218617	0.401272627967599	0.466733533781383
After testing	0.999999999994986	0.999999999993743	5.01410024611459E-12

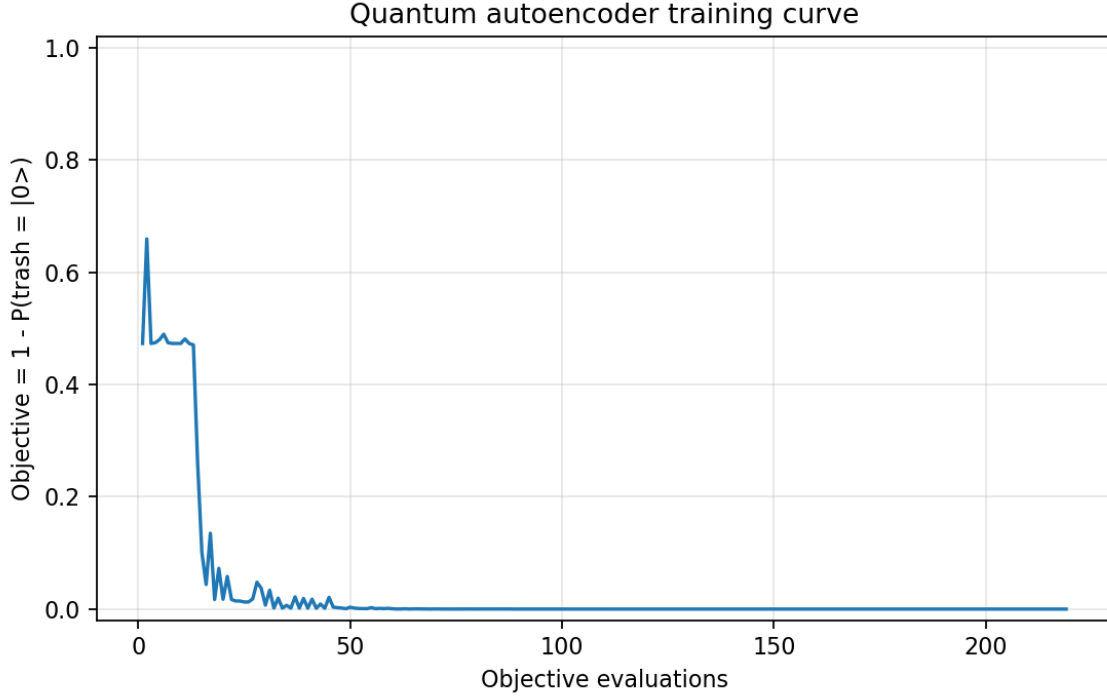


Figure 4: Training Curve : Objective vs iteration

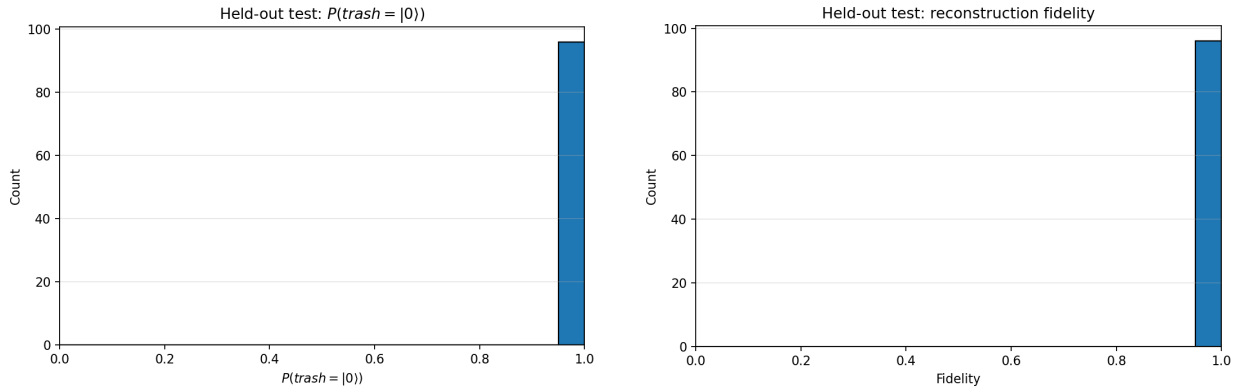


Figure 5: Evaluation plots

6 Classical Comparison

For classical data, the analogy is straightforward: the classical equivalent is the standard autoencoder (or linear PCA in the simplest case). Classical deep autoencoders are a mature tool for nonlinear dimensionality reduction and have decades of optimization and regularization practice behind them. The classic Science paper by Hinton & Salakhutdinov is one of the canonical references demonstrating that deep autoencoders can learn low-dimensional codes that outperform PCA for certain tasks. [5]

For quantum data, the conceptual classical analogue is not a neural network; it is quantum source coding (Schumacher compression). Schumacher’s noiseless coding theorem gives the asymptotic compressibility of a quantum source in terms of von Neumann entropy. There are explicit reversible algorithms for Schumacher compression, which reinforces an important point: “quantum compression” is not inherently a machine-learning problem; it becomes one when you want a practical, data-driven way to discover a good compressor for a structured family of states using only access to example states. [12, 1]

7 Limitations and Challenges

Near-term feasibility is bounded by NISQ constraints: limited qubit counts, restricted circuit depth due to noise, and sampling overhead for estimating losses and gradients. [2]

Trainability can be a first-order limitation: barren plateaus imply gradients can become exponentially small as qubit counts increase for broad classes of ansätze and cost functions, and the effect depends strongly on whether the cost is global or local. [10, 11]

For classical-data variants, state preparation and input models are often the bottleneck; amplitude-style loading can be expensive for generic data, and proposed mitigations depend on assumptions about data access and robustness. [6, 7]

8 Critical Evaluation

A QAE can be a sensible research tool when the data are quantum-native and compression reduces scarce quantum resources within a broader quantum workflow. For general-purpose classical ML pipelines, the overheads (encoding, noise, sampling, and training instability) typically dominate and make QAEs hard to justify today. [1, 2]

Where I would consider a QAE seriously is quantum-native workflows, where the “data” are intermediate quantum states you already have on a device and you’re bottlenecked by qubit count, communication bandwidth, or memory. In that domain, “compress a family of quantum states without full tomography” is a legitimate operational goal, and QAE-style learning can be a practical heuristic. That aligns with the original motivation and experimental demonstrations that emphasize minimal prior information about the dataset structure. [1]

There is also a promising but still unsettled niche around using QAEs for denoising or error mitigation. Some work explicitly argues for quantum autoencoders capable of quantum advantage for denoising quantum data, and others propose detection-based mitigation strategies that use an autoencoder-like compression to detect errors. These are interesting directions, but they are not yet “drop-in” replacements for established error-correction frameworks—and their performance is tightly coupled to noise models and scaling behaviour. [4]

9 Quantum Advantage Discussion

The strongest case for quantum advantage is when the objects being compressed are quantum states that are not efficiently representable or manipulable classically; in that regime, the QAE is not competing with a classical autoencoder on equal input access. [1]

For classical-data QAEs, any advantage is conditional on input models, state-preparation costs, and trainability under realistic noise and sampling constraints. Theoretical analyses of state preparation (and counter-analyses under smoothed robustness assumptions) highlight why “advantage” is often unclear without careful accounting. [6, 7, 2]

10 Conclusion

Quantum autoencoders are conceptually simple but practically nuanced. The core idea—learn a unitary that pushes redundant information into a trash subsystem that can be reset/discarded—connects quantum information theory (compression) with variational learning (trainable circuits). In practice, quantum autoencoders inherit the full set of limitations associated with near-term quantum hardware: noise, limited circuit depth, finite measurement shots. Training can become expensive and unreliable as system size grows, and this quickly weakens the practical advantage of the method. For workflows built from classical data, the cost of encoding that data into quantum states often outweighs any benefit gained from quantum compression. QAE’s are more of an experimental model rather than a working product - with current available hardware. [1, 2]

References

- [1] Jonathan Romero, Jonathan P. Olson, and Alán Aspuru-Guzik. “Quantum autoencoders for efficient compression of quantum data”. In: *Quantum Science and Technology* 2.4 (2017), p. 045001. DOI: [10.1088/2058-9565/aa8072](https://doi.org/10.1088/2058-9565/aa8072). arXiv: [1612.02806](https://arxiv.org/abs/1612.02806).
- [2] John Preskill. “Quantum Computing in the NISQ era and beyond”. In: *Quantum* 2 (2018), p. 79. DOI: [10.22331/q-2018-08-06-79](https://doi.org/10.22331/q-2018-08-06-79). arXiv: [1801.00862](https://arxiv.org/abs/1801.00862).
- [3] Chang-Jiang Huang, Hailan Ma, Qi Yin, Jun-Feng Tang, Daoyi Dong, Chunlin Chen, Guo-Yong Xiang, Chuan-Feng Li, and Guang-Can Guo. “Realization of a quantum autoencoder for lossless compression of quantum data”. In: *Physical Review A* 102.3 (2020), p. 032412. DOI: [10.1103/PhysRevA.102.032412](https://doi.org/10.1103/PhysRevA.102.032412). arXiv: [1903.08699](https://arxiv.org/abs/1903.08699).
- [4] Dmytro Bondarenko and Polina Feldmann. “Quantum autoencoders to denoise quantum data”. In: *Physical Review Letters* 124.13 (2020), p. 130502. DOI: [10.1103/PhysRevLett.124.130502](https://doi.org/10.1103/PhysRevLett.124.130502). arXiv: [1910.09169](https://arxiv.org/abs/1910.09169).
- [5] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. “Reducing the Dimensionality of Data with Neural Networks”. In: *Science* 313.5786 (2006), pp. 504–507. DOI: [10.1126/science.1127647](https://doi.org/10.1126/science.1127647).
- [6] Mikko Möttönen, Juha J. Vartiainen, Ville Bergholm, and Martti M. Salomaa. *Transformation of quantum states using uniformly controlled rotations*. arXiv:quant-ph/0407010. 2004. arXiv: [quant-ph/0407010](https://arxiv.org/abs/quant-ph/0407010).
- [7] Zhikuan Zhao, Jack K. Fitzsimons, Patrick Rebentrost, Vedran Dunjko, and Joseph F. Fitzsimons. “Smooth input preparation for quantum and quantum-inspired machine learning”. In: *Quantum Machine Intelligence* 3.1 (2021), p. 14. DOI: [10.1007/s42484-021-00045-x](https://doi.org/10.1007/s42484-021-00045-x). arXiv: [1804.00281](https://arxiv.org/abs/1804.00281).
- [8] Alex Pepper, Nora Tischler, and Geoff J. Pryde. “Experimental Realization of a Quantum Autoencoder: The Compression of Qutrits via Machine Learning”. In: *Physical Review Letters* 122.6 (2019), p. 060501. DOI: [10.1103/PhysRevLett.122.060501](https://doi.org/10.1103/PhysRevLett.122.060501). arXiv: [1810.01637](https://arxiv.org/abs/1810.01637).
- [9] Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. “Evaluating analytic gradients on quantum hardware”. In: *Physical Review A* 99.3 (2019), p. 032331. DOI: [10.1103/PhysRevA.99.032331](https://doi.org/10.1103/PhysRevA.99.032331). arXiv: [1811.11184](https://arxiv.org/abs/1811.11184).
- [10] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. “Barren plateaus in quantum neural network training landscapes”. In: *Nature Communications* 9 (2018), p. 4812. DOI: [10.1038/s41467-018-07090-4](https://doi.org/10.1038/s41467-018-07090-4). arXiv: [1803.11173](https://arxiv.org/abs/1803.11173).
- [11] M. Cerezo, Akira Sone, Tyler Volkoff, Lukasz Cincio, and Patrick J. Coles. “Cost function dependent barren plateaus in shallow parametrized quantum circuits”. In: *Nature Communications* 12 (2021), p. 1791. DOI: [10.1038/s41467-021-21728-w](https://doi.org/10.1038/s41467-021-21728-w). arXiv: [2001.00550](https://arxiv.org/abs/2001.00550).
- [12] Benjamin Schumacher. “Quantum coding”. In: *Physical Review A* 51.4 (1995), pp. 2738–2747. DOI: [10.1103/PhysRevA.51.2738](https://doi.org/10.1103/PhysRevA.51.2738).